PROJECT DISCOVERY & REQUIREMENTS

# Semantic Analysis in Machine Learning

*Questions to ask an instructor before research, architecture, and development*

**Core principle: “Build an application based on semantic analysis” is not yet a sufficient project specification. Before selecting a model, dataset, framework, or architecture, define the problem, users, input, output, constraints, and success criteria.**

Prepared as a project-discovery questionnaire

*Use this document as the question checklist before starting research work.*

# How to use this document

Treat the questions as a requirements interview. Do not try to answer every question blindly: questions that do not apply should be marked N/A. The objective is to remove ambiguity before technical decisions are made.

## Three levels of questioning

| Level | Focus |
|---|---|
| **Level 1 - What are we building?** | Problem, users, input, output, domain, and objective. |
| **Level 2 - How should we build it?** | Dataset, preprocessing, algorithms, embeddings, model architecture, evaluation, backend, frontend, and database. |
| **Level 3 - What constraints must we obey?** | Deadline, budget, hardware, allowed libraries/APIs, deployment, security, academic rules, and deliverables. |

## Important warning

Choosing the model before defining the problem is backwards. “Should we use BERT?” is a technical question; “What exact problem should the system solve?” is a project-definition question. The second one determines whether the first one even matters.

## 1. First: What exactly is the application?

1. What is the exact problem this application is supposed to solve?
2. What is the application supposed to do with text?
3. What do you mean specifically by “semantic analysis” in this project?
4. Are you expecting semantic similarity, sentiment analysis, intent classification, topic classification, semantic search, question answering, text summarization, named entity recognition, emotion detection, document classification, semantic parsing, or multiple tasks?
5. What is the expected input?
6. What is the expected output?
7. Can you give me 3-5 real examples of input and expected output?
8. Who is going to use the application?
9. What real-world problem does it solve for those users?
10. What makes this different from simply using ChatGPT, Google, or an existing NLP API?
11. Is this supposed to be a research project, academic project, prototype, hackathon project, or production application?
12. What is the minimum functionality that must work for the project to be considered complete?

## 2. Project objective

13. What is the primary objective of the project?
14. What are the secondary objectives?
15. What problem should the machine-learning component solve?
16. What decisions should the application make automatically?
17. What should the system merely suggest to the user instead of deciding automatically?
18. What is considered a successful outcome?
19. Are there measurable project goals?
20. Do you expect improvement over an existing method or baseline?
21. Is the goal to demonstrate the concept or achieve high real-world accuracy?

## 3. Target users

22. Who are the intended users?
23. Are they students, teachers, businesses, researchers, customers, administrators, developers, etc.?
24. How technically skilled are the users?
25. How frequently will they use the system?
26. Will there be multiple types of users?
27. Does each user type require different features?
28. Do users need accounts?
29. Do different users need different permissions?
30. Is the application intended for a specific industry or domain?

## 4. Domain

31. What domain should the application focus on?
32. Is the domain general-purpose or domain-specific?
33. What type of language will the system analyze?
34. Is the text formal or informal?
35. Will the text contain technical terminology?
36. Will the system handle slang?
37. Will it handle abbreviations?
38. Will it handle spelling mistakes?
39. Will it handle mixed languages?
40. Will it handle Indian languages?
41. Will it handle Tamil/English or Tanglish?
42. Is domain-specific vocabulary expected?
43. Are there domain-specific rules we need to incorporate?

**Example:** semantic analysis of student answers, customer reviews, legal documents, medical documents, cybersecurity reports, or news articles can require very different datasets, preprocessing, models, and evaluation methods.

## 5. Input requirements

44. What format will users provide: plain text, PDF, Word documents, CSV, URLs, images, scanned documents, emails, chat messages, or audio converted to text?
45. Is the input one sentence, paragraph, document, or thousands of documents?
46. What is the maximum input length?
47. Can users upload multiple files?
48. Should the application process documents automatically?
49. Should OCR be required?
50. Should speech-to-text be included?
51. What languages should input support?
52. Should the system preserve formatting?
53. Can the input contain tables?
54. Can the input contain code?
55. Can the input contain URLs?
56. Should irrelevant content be removed before analysis?

## 6. Output requirements

57. What exactly should the system display after analysis?
58. Should it produce a classification?
59. Should it produce a similarity score?
60. Should it produce keywords?
61. Should it extract entities?
62. Should it provide a summary?
63. Should it provide explanations?

64. Should it highlight important sentences?
65. Should it show confidence scores?
66. Should it show the reasoning behind the classification?
67. Should it generate recommendations?
68. Should the output be downloadable?
69. Should results be stored?
70. Should users be able to compare two texts?
71. Should users be able to search previously analyzed documents?

## 7. Semantic-analysis definition

72. What does “semantic” mean in the context of this project?
73. Are we interested in the meaning of individual words?
74. Meaning of sentences?
75. Meaning of entire documents?
76. Relationships between words?
77. Relationships between sentences?
78. Similarity between documents?
79. User intent?
80. Context?
81. Sentiment?
82. Topic?
83. Entities and relationships?
84. Logical relationships?

**Concrete example:** “The laptop battery is terrible.” and “My laptop loses power very quickly.” use different words but can express related meaning. Clarify whether recognizing this type of relationship is part of the requested system.

## 8. Machine-learning requirements

85. Are we required to use machine learning?
86. Is deep learning allowed?
87. Are pretrained models allowed?
88. Can we use transformers?
89. Can we use BERT?
90. Can we use RoBERTa?
91. Can we use Sentence-BERT?
92. Can we use Word2Vec?
93. Can we use GloVe?
94. Can we use TF-IDF as a baseline?
95. Can we use large language models?
96. Can we use OpenAI/Gemini/Claude APIs?
97. Are external APIs prohibited?
98. Must the model be trained from scratch?
99. Must we fine-tune an existing model?

100. Can we use Hugging Face models?
101. Can we use cloud-based models?
102. Must inference run locally?
103. Does the instructor expect us to implement part of the algorithm ourselves?

## 9. Dataset

104. Where will the dataset come from?
105. Do you already have a dataset?
106. If not, are we expected to collect one?
107. Can we use public datasets?
108. Which datasets are permitted?
109. Are datasets from Kaggle allowed?
110. Are datasets from Hugging Face allowed?
111. Can we scrape websites?
112. Can we collect data through APIs?
113. How many records are expected?
114. What languages should the dataset contain?
115. Is the dataset labeled?
116. Who provides the labels?
117. How reliable are those labels?
118. Do we need manual annotation?
119. How many classes exist?
120. Is the dataset balanced?
121. How will missing values be handled?
122. How will duplicate data be handled?
123. What is the expected train/validation/test split?
124. Can synthetic data be used?
125. Can generative AI be used to create training data?

## 10. Data preprocessing

126. What preprocessing steps are expected?
127. Should we remove stop words?
128. Should we perform stemming?
129. Should we perform lemmatization?
130. Should punctuation be removed?
131. Should emojis be retained?
132. Should hashtags be retained?
133. Should URLs be removed?
134. Should spelling correction be performed?
135. Should lowercase conversion be used?
136. How should numbers be handled?
137. How should abbreviations be handled?
138. How should special characters be handled?
139. How should multilingual text be handled?

## 11. Model architecture

140. Do you expect a traditional ML model or deep-learning model?
141. Which algorithms are acceptable?
142. Should we compare multiple models?
143. Should we establish a baseline model?
144. Should we experiment with multiple embeddings?
145. Should we use word embeddings or sentence embeddings?
146. Should we fine-tune a transformer?
147. Should the model include multiple stages?
148. Do we need a custom semantic-analysis pipeline?
149. Does the instructor expect an architectural diagram?
150. Are explainable-AI techniques required?

## 12. Accuracy and evaluation

151. How will we decide whether the model works?
152. What metric should be used?
153. Accuracy?
154. Precision?
155. Recall?
156. F1-score?
157. ROC-AUC?
158. Mean squared error?
159. Cosine similarity?
160. Pearson/Spearman correlation?
161. BLEU/ROUGE for generated text?
162. What minimum performance is expected?
163. Should we compare our model against a baseline?
164. Should we conduct error analysis?
165. How many test cases are required?
166. Should human evaluation be performed?
167. How should false positives be handled?
168. How should false negatives be handled?

**Do not choose evaluation metrics by habit.** The metric must match the task. For example, classification, ranking, semantic similarity, and generation require different evaluation approaches.

## 13. Explainability

169. Does the user need to know why the model produced its result?
170. Should the application highlight the words/sentences that influenced the result?
171. Should attention or feature importance be shown?
172. Should the system provide human-readable explanations?
173. Is explainable AI a project requirement?

174. How detailed should the explanation be?

## 14. Application interface

175. What platform should the application run on: Web, Desktop, Mobile, CLI, or API only?
176. Do you expect a frontend?
177. Should the frontend be simple or production-like?
178. Are there required technologies such as HTML/CSS/JavaScript, React, Django, Flask, FastAPI, or Streamlit?
179. Should the application be responsive?
180. Should users upload files through the interface?
181. Should analysis happen in real time?
182. Should progress indicators be shown?
183. Should results be displayed as charts?
184. Should results be exported as PDF/CSV?
185. Should users have dashboards?
186. Should there be a history page?

**Possible platforms:** Web, desktop, mobile, command-line, or API-only. The interface choice depends on who uses the system and how results must be consumed.

## 15. Backend

187. What backend technology is allowed?
188. Do we need REST APIs?
189. Is authentication required?
190. Is authorization required?
191. What database should be used?
192. What information should be stored?
193. Should analyzed text be stored?
194. Should model results be stored?
195. Should user history be stored?
196. How long should data be retained?
197. Should there be an admin panel?
198. Do we need background processing for large documents?

## 16. Database

199. Do we need a database?
200. Which database is preferred: PostgreSQL, MySQL, MongoDB, or SQLite?
201. What entities need to be stored?
202. Do we need document versioning?
203. Do we need analysis history?
204. Do we need user profiles?
205. Do we need model-version tracking?
206. Do we need audit logs?

## 17. Deployment

207. Where should the application run?
208. Local machine?
209. College server?
210. Cloud?
211. Docker container?
212. Which cloud providers are allowed?
213. Is deployment mandatory?
214. Is a public URL required?
215. How many simultaneous users are expected?
216. What hardware is available?
217. Is GPU access available?
218. How much RAM/CPU is available?
219. What is the acceptable response time?

## 18. Security and privacy

220. Can users upload confidential information?
221. Is personal information involved?
222. Is sensitive information involved?
223. Should uploaded documents be encrypted?
224. Should data be deleted after analysis?
225. Can data be sent to external APIs?
226. Are third-party AI APIs permitted?
227. Are there restrictions on storing user text?
228. Do we need authentication?
229. Do we need role-based access control?
230. Do we need audit logs?
231. What security standards are expected?

Privacy matters particularly when uploaded documents may contain confidential, personal, or sensitive information. Confirm whether data can leave the organization or be sent to third-party APIs.

## 19. Research requirements

232. Is this project supposed to contain a research component?
233. Do you expect a literature review?
234. How many research papers should be studied?
235. Which publication years should we consider?
236. Are IEEE papers required?
237. Are Springer/Elsevier/ACM papers acceptable?
238. Do you require recent papers?
239. Should we identify a research gap?
240. Are we expected to reproduce an existing methodology?

241. Are we expected to propose a new methodology?
242. Should we compare existing techniques?
243. Should the research produce an experimental result?
244. Is publication a goal?

## 20. Existing systems

245. What existing applications should we study?
246. Which existing solutions are considered competitors or baselines?
247. What limitations do those systems have?
248. Are we expected to solve a known limitation?
249. Is there an existing application that inspired this project?
250. Are there existing papers whose approach we should follow?
251. Are there specific technologies you do not want us to use?

## 21. Project scope

252. How large should the project be?
253. What features are mandatory?
254. What features are optional?
255. What features are explicitly out of scope?
256. Is this a semester project?
257. How much development time do we have?
258. How many developers are working on it?
259. What skill level does the team have?
260. What resources are available?
261. What is the budget?

## 22. Deliverables

262. What exactly do you expect us to submit: source code, research paper, project report, presentation, architecture diagram, UML diagrams, dataset, trained model, API, frontend, deployment, or demo video?
263. Do we need documentation?
264. Do we need a README?
265. Do we need installation instructions?
266. Do we need an API document?
267. Do we need test cases?
268. Do we need a user manual?
269. Do we need a research paper?
270. Do we need a presentation/demo?

## 23. Academic constraints

271. Are external libraries allowed?
272. Are pretrained models allowed?
273. Are AI tools allowed?

274. Is ChatGPT/Copilot allowed?
275. Can we use code from GitHub?
276. Can we use open-source models?
277. Are citations required for implementation ideas?
278. Is plagiarism checking performed?
279. Are there restrictions on copying architectures from papers?
280. Does the project need original work?

## 24. Testing

281. How should the application be tested?
282. Do you require unit testing?
283. Integration testing?
284. Model testing?
285. UI testing?
286. Load testing?
287. Security testing?
288. How many test cases are required?
289. Should edge cases be tested?
290. What happens when the model receives meaningless text?
291. What happens when the input is extremely long?
292. What happens when the input is in an unsupported language?
293. What happens when the model is uncertain?

## 25. Failure handling

294. What should happen when the model cannot determine meaning?
295. Should the application return “unknown”?
296. Should it ask the user for clarification?
297. Should low-confidence predictions be rejected?
298. Should users be able to correct the result?
299. Can those corrections be used as future training data?

## 26. Scalability

300. How many documents might eventually be processed?
301. Does the system need batch processing?
302. Does it need real-time processing?
303. Could multiple users analyze documents simultaneously?
304. Does the system need asynchronous processing?
305. Should the ML model be loaded once or per request?
306. Will model inference become a performance bottleneck?

## 27. Future scope

307. Is this intended to become a larger system later?
308. Should the architecture support additional models?

309. Should new languages be added later?
310. Should additional semantic-analysis features be added later?
311. Should the system eventually support multimodal data?
312. Should it eventually support images/audio/video?
313. Should users eventually be able to train custom models?

## 28. The questions I would consider absolutely mandatory

314. What problem are we solving?
315. Who uses it?
316. What is the input?
317. What is the output?
318. What domain are we working in?
319. What machine-learning/NLP task is required?
320. What dataset can we use?
321. What model or modeling approach is permitted/expected?
322. How will we evaluate it?
323. What interface is expected?
324. What backend/database is expected?
325. Where will it be deployed?
326. What is the deadline?
327. What restrictions apply?
328. What deliverables are required?

## Mandatory project-definition checklist

| Question | What must be clarified |
|---|---|
| **What problem?** | Exact problem the application solves. |
| **Who uses it?** | Target users and their context. |
| **Input?** | Text/document format, size, languages. |
| **Output?** | Exact result shown to the user. |
| **Domain?** | General or domain-specific language. |
| **ML task?** | Similarity, classification, intent, sentiment, search, etc. |
| **Dataset?** | Source, labels, size, permissions. |
| **Model?** | Baseline, pretrained model, fine-tuning, local/API constraints. |
| **Evaluation?** | Metrics, test strategy, minimum expectations. |
| **Interface?** | Web/mobile/desktop/API and required frameworks. |
| **Backend/database?** | Architecture and data-storage requirements. |
| **Deployment?** | Local/cloud/server, hardware, performance targets. |
| **Deadline?** | Available time and milestones. |
| **Restrictions?** | Tools, libraries, APIs, originality, academic rules. |
| **Deliverables?** | Code, report, paper, demo, documentation, deployment. |

## The first conversation I would have with the instructor

> **“Sir/Madam, when you say ‘build an application based on semantic analysis in machine learning,’ what exact problem do you want the system to solve? Please give me one example of an input and the output you expect.”**
>
> **Then:** “Who is the target user, what type of text will be analyzed, what semantic task is required, what dataset can we use, what technologies are permitted, what are the project constraints, and what exactly do you expect as the final deliverable?”

## Final principle

A project brief that only says “semantic analysis in machine learning” is incomplete. The right first step is not model selection; it is requirements discovery. Once the problem, user, input, output, dataset, constraints, evaluation criteria, and deliverables are clear, the research direction and technical architecture become much easier to justify.